UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

--------***--------

REPORT LABWORK 2

MACHINE LEARNING AND DATA MINING II

CLASS: 1

	STUDENT NAME	STUDENT ID
1	Nguyễn Quang Hưng	22BI13184
2	Vũ Tuấn Hải	22BI13149

Ha Noi, February 18th 2025

For this lab work, we selected three datasets from the UCI repository. The first two datasets are the Zoo dataset and the Automobile dataset. The third dataset is the Crime dataset.

K-means

I. Zoo dataset

1. Dataset

1.1. Dataset Overview

The dataset is read and converted into a DataFrame using Pandas. The data is accessed directly through a URL.

```
# Load dataset
zoo_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/zoo/zoo.data'
zoo_columns = ["animal_name", "hair", "feathers", "eggs", "milk", "airborne",
               "aquatic", "predator", "toothed", "backbone", "breathes", "venomous",
               "fins", "legs", "tail", "domestic", "catsize", "type"]
zoo_data = pd.read_csv(zoo_url, names=zoo_columns)
print(zoo_data.head())
print("\nDataset shape:", zoo_data.shape)
```

Our original dataset includes *101 instances* and compares *18 features*.

1.2. Dataset Processing

Before applying clustering, we need to preprocess the dataset to ensure it is suitable for the clustering algorithm.

a. Data format

```
# Select features for clustering
X_zoo = zoo_data.drop(["animal_name", "type"], axis=1) # remove 'animal_name' and 'type'
print(X_zoo.head())
print("\nFeatures shape:", X_zoo.shape)
```

- The dataset is in tabular form, where columns represent features. We clean the data by removing missing values and eliminating non-essential attributes such as animal_name (not useful for clustering) and type (class label). After preprocessing, the dataset retains 16 features.
- After dropping, the dataset retains *16 features*.

b. Standardization

K-Means is sensitive to distance metrics. Without standardization, differences in feature scales could disproportionately influence the clustering results.

We use *StandardScaler* here to make the features have the same scale.

```
# Standardization
scaler_zoo = StandardScaler()
X_zoo_scaled = scaler_zoo.fit_transform(X_zoo)
print(X_zoo_scaled[:5])
```

After applying standardization, we obtain transformed features as follows:

```
[[ 1.16139451 -0.49690399 -1.18522652  1.20971676 -0.55829053 -0.74420841
   0.89642146  0.80977633  0.46569032  0.51234754 -0.29329423 -0.44986771
   0.57253971 -1.69841555 -0.38435306  1.13818037]
 [ 1.16139451 -0.49690399 -1.18522652  1.20971676 -0.55829053 -0.74420841
  -1.1155467   0.80977633  0.46569032  0.51234754 -0.29329423 -0.44986771
   0.57253971  0.58878406 -0.38435306  1.13818037]
 [-0.86103386 -0.49690399  0.84372057 -0.82663978 -0.55829053  1.34370962
   0.89642146  0.80977633  0.46569032 -1.95180015 -0.29329423  2.22287572
  -1.40443503  0.58878406 -0.38435306 -0.87859537]
 [ 1.16139451 -0.49690399 -1.18522652  1.20971676 -0.55829053 -0.74420841
   0.89642146  0.80977633  0.46569032  0.51234754 -0.29329423 -0.44986771
   0.57253971 -1.69841555 -0.38435306  1.13818037]
 [ 1.16139451 -0.49690399 -1.18522652  1.20971676 -0.55829053 -0.74420841
   0.89642146  0.80977633  0.46569032  0.51234754 -0.29329423 -0.44986771
   0.57253971  0.58878406 -0.38435306  1.13818037]]
```

2. Run experiments with k-means

```
# K-Means clustering for k from 2 to 10
ks = range(2, 11)
inertias_zoo = []
silhouettes_zoo = []
```

- We test K values ranging from 2 to 10, initializing empty lists to record inertia (sum of squared intra-cluster distances) and silhouette scores.

```
print("K-Means Clustering")
for k in ks:
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    labels = kmeans.fit_predict(X_zoo_scaled)

   # Calculate inertia
    inertia = kmeans.inertia_

    # Calculate silhouette score, handle any ValueError in case a cluster has only 1 sample
    try:
        silhouette_avg = silhouette_score(X_zoo_scaled, labels)
    except ValueError as e:
        silhouette_avg = np.nan
        print(f"[Warning] k={k} - Error computing silhouette_score: {e}")

    inertias_zoo.append(inertia)
    silhouettes_zoo.append(silhouette_avg)
    print(f"k = {k}: Inertia = {inertia:.2f}, Silhouette Score = {silhouette_avg}")
```

- From *scikit-learn*, we use the `init='k-means++'` parameter. The k-means++ algorithm selects the first cluster centroid randomly, then chooses subsequent centroids with a probability proportional to the squared distance from the nearest selected centroid.
- Then, we use fit_predict to fit the models `X_zoo_scaled` and return cluster labels.
- We calculate the clustering quality including Inertia and Silhouette score based on k-means, there is also a function to check errors if clusters with a single point.

```
K-Means Clustering
k = 2: Inertia = 1177.50, Silhouette Score = 0.27804550249405496
k = 3: Inertia = 927.27, Silhouette Score = 0.34023031976791035
k = 4: Inertia = 720.76, Silhouette Score = 0.3919582327156801
k = 5: Inertia = 647.51, Silhouette Score = 0.3728875866543154
k = 6: Inertia = 581.55, Silhouette Score = 0.3964098976350089
k = 7: Inertia = 529.40, Silhouette Score = 0.3695899977921989
k = 8: Inertia = 490.63, Silhouette Score = 0.3487575813490419
k = 9: Inertia = 432.83, Silhouette Score = 0.3909627596533814
k = 10: Inertia = 395.90, Silhouette Score = 0.40553785598244496
```

After the run, we gain the above values of k-means.

3. Comparison

We use values that are computed above to plot a graph comparing Inertia and Silhouette Scores:

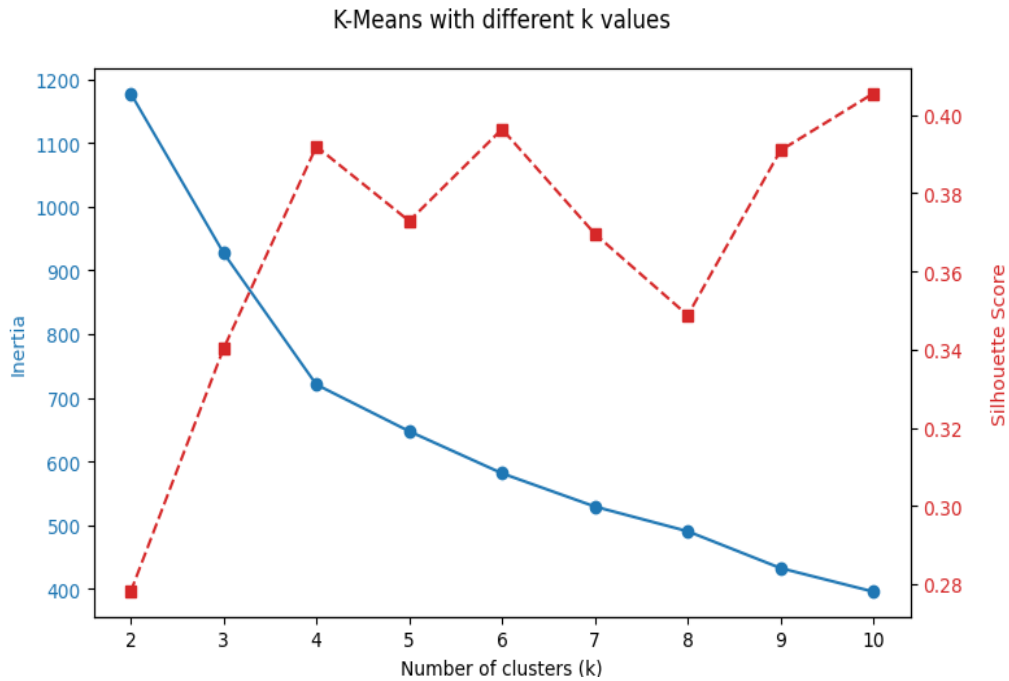
```
# Plot a comparison
fig, ax1 = plt.subplots(figsize=(8, 5))

color = 'tab:blue'
ax1.set_xlabel('Number of clusters (k)')
ax1.set_ylabel('Inertia', color=color)
ax1.plot(ks, inertias_zoo, marker='o', color=color, label="Inertia")
ax1.tick_params(axis='y', labelcolor=color)
ax1.set_title("K-Means with different k values\n")

ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('\nSilhouette score', color=color)
ax2.plot(ks, silhouettes_zoo, marker='s', linestyle='--', color=color, label="Silhouette Score")
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()
plt.show()
```

Output:



II. Automobile dataset

1. Dataset

1.1. Dataset Overview

The data will be read and converted into a dataframe using pandas, and will be read directly through an URL.

```
# Load dataset
auto_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data"
auto_columns = ["symboling", "normalized-losses", "make", "fuel-type", "aspiration",
                "num-of-doors", "body-style", "drive-wheels", "engine-location", "wheel-base",
                "length", "width", "height", "curb-weight", "engine-type", "num-of-cylinders",
                "engine-size", "fuel-system", "bore", "stroke", "compression-ratio",
                "horsepower", "peak-rpm", "city-mpg", "highway-mpg", "price"]

auto_data = pd.read_csv(auto_url, names=auto_columns, na_values="?")
print(auto_data.head())
print("Dataset shape:", auto_data.shape)
```

This original dataset contains *205 instances* and *26 features*.

1.2. Dataset Processing

We proceeded to alter the values to suit the clustering model.

a. Data format

We deleted missing values and non-predictive columns

```
# Select features for clustering
numeric_cols = auto_data.select_dtypes(include=[np.number]).columns
X_auto = auto_data[numeric_cols].dropna()  # remove missing value
print(X_auto.head())
print("\nDataset shape:", X_auto.shape)
```

After processing, the values of instances và features have reduced to 160 và 16.

b. Standardization

With the same process as the Zoo dataset, including the use of *StandardScaler*

```
# Standardization
scaler_auto = StandardScaler()
X_auto_scaled = scaler_auto.fit_transform(X_auto)
print(X_auto_scaled[:5])
```

We get the following result:

```
[[ 1.06469263  1.20312243  0.30390372  0.37181722  0.31106628  0.18561505
  -0.25542735 -0.33295123 -0.40687794  0.55470043 -0.03749655  0.20062331
   0.827343   -0.41342425 -0.32219564  0.43150225]
 [ 1.06469263  1.20312243  0.22619761  0.37181722  0.41411101  0.18561505
   0.76044133  0.55766754 -0.40687794  0.55470043 -0.55424494  0.62643605
   0.827343   -1.4031681  -1.56814855  1.03025998]
 [ 0.22137173  1.03406542  1.46949528  1.77027125  2.99022929  0.80249591
   0.80216078  0.55766754 -0.40687794  0.55470043 -0.42505784  0.46266192
   0.827343   -1.23821079 -1.10091621  1.07473913]
 [ 0.22137173  1.03406542  1.46949528  1.77027125  2.99022929  0.89062175
   1.30696616  0.39273814 -0.63200925  0.55470043 -0.47673268  1.4453067
   0.827343   -1.5681254  -1.87963678  2.12940812]
 [ 1.06469263  1.99205518  0.57587508  0.38918932 -0.41024684  0.18561505
  -0.13444094 -0.36593711  0.75630049 -1.49106373 -0.34754558  0.16786848
   1.47412581 -0.57838156 -0.47793975  0.85576487]]
```

2. Run experiments with k-means

This time, the clustering range will also be from 2 to 10

```
# K-Means clustering for k from 2 to 10
ks = range (2, 10)
inertias_auto = []
silhouettes_auto = []
```

```
print("K-Means Clustering")
for k in ks:
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    labels = kmeans.fit_predict(X_auto_scaled)
    inertia = kmeans.inertia_
    silhouette_avg = silhouette_score(X_auto_scaled, labels)
    inertias_auto.append(inertia)
    silhouettes_auto.append(silhouette_avg)
    print(f"k = {k}: Inertia = {inertia:.2f}, Silhouette Score = {silhouette_avg:.3f}")
```

- From *scikit-learn*, we apply the init='k-means++' parameter. The k-means++ algorithm selects the first cluster centroid randomly, then chooses subsequent centroids with a probability proportional to the squared distance from the nearest selected centroid.
- Then, we use fit_predict to fit the models X_auto_scaled and return cluster labels.
- We calculate the clustering quality based on k-means.

The k-means will be returned as follows:

```
K-Means Clustering
k = 2: Inertia = 1674.34, Silhouette Score = 0.330
k = 3: Inertia = 1443.83, Silhouette Score = 0.204
k = 4: Inertia = 1260.24, Silhouette Score = 0.210
k = 5: Inertia = 1128.25, Silhouette Score = 0.223
k = 6: Inertia = 1045.02, Silhouette Score = 0.253
k = 7: Inertia = 925.98, Silhouette Score = 0.262
k = 8: Inertia = 854.50, Silhouette Score = 0.283
k = 9: Inertia = 819.38, Silhouette Score = 0.247
k = 10: Inertia = 743.53, Silhouette Score = 0.266
```

3. Comparison

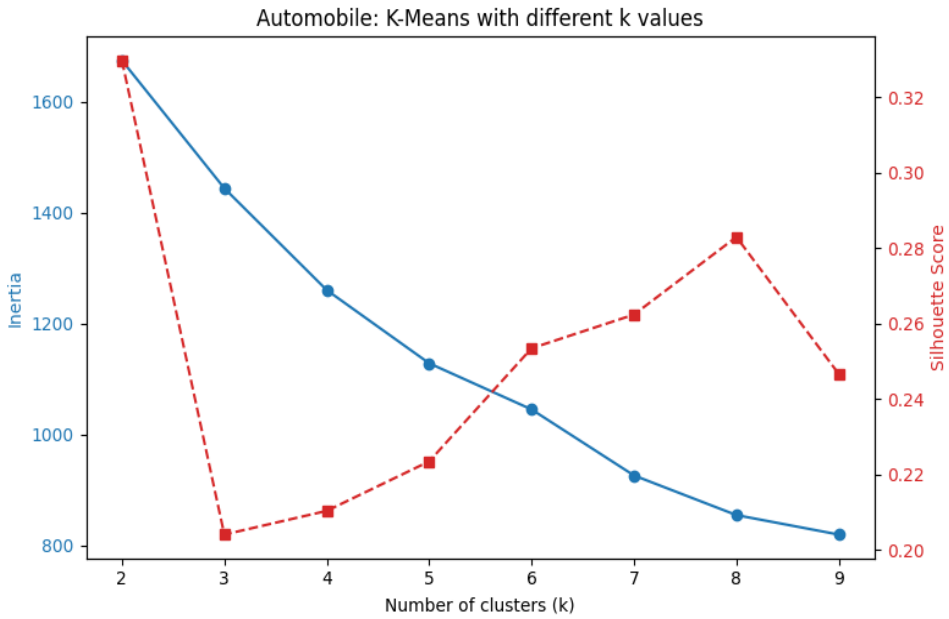
We use the computed values above to plot a graph comparing Inertia and Silhouette Scores:

```
# Plot a comparision
fig, ax1 = plt.subplots(figsize=(8, 5))
color = 'tab:blue'
ax1.set_xlabel('Number of clusters (k)')
ax1.set_ylabel('Inertia', color=color)
ax1.plot(ks, inertias_auto, marker='o', color=color, label="Inertia")
ax1.tick_params(axis='y', labelcolor=color)
ax1.set_title("Automobile: K-Means with different k values")

ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Silhouette Score', color=color)
ax2.plot(ks, silhouettes_auto, marker='s', linestyle='--', color=color, label="Silhouette Score")
ax2.tick_params(axis='y', labelcolor=color)

fig.tight_layout()
plt.show()
```

After running, we have a graph that depicts K-means with differing numbers of clusters:



Crime Dataset

I. Dataset processing

1. Original dataset

We use pandas to read and convert the dataset that takes directly through URL into a dataframe

```
# Use the dataset URL from UCI for the Communities and Crime dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/communities/communities.data"
df = pd.read_csv(url, header=None, na_values='?')

print(df.head())
print("Original dataset shape:", df.shape)
```

The original dataset contains *1994 instances* and *128 features*.

2. Dataset after processing

- There are missing values (NaN) in the table, so we replaced those values with the Median. We also converted all the data to numeric.

```
# Filter the feature for clustering
df = df.iloc[:, 5:-1]
df = df.apply(pd.to_numeric, errors='coerce')
df.fillna(df.median(), inplace=True)
print(df.head())
print("\nDataset shape after processing:", df.shape)
```

In the table after processing, 104 values were remaining.

- We continuously used *StandardScaler* to standardize the table above.

```
# Standardize the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(df)
print(data_scaled[:5])
```

Output:

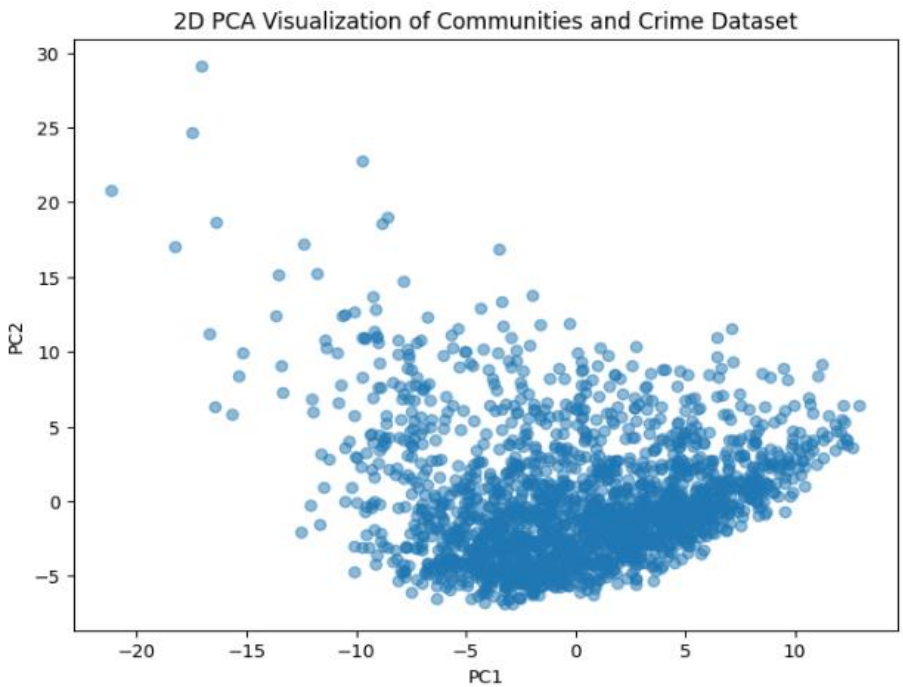
```
[[ 5.85942368e-01 -1.04354503e+00 -7.55482476e-01 -2.93978258e-01
   7.22626691e-02  2.60384663e-01  1.17536782e-01  1.68473562e-01
   4.03647454e-01 -2.68000463e-01  3.94291607e-01  1.29599273e-01
   1.91491812e-01 -4.94814960e-01 -1.01172665e+00 -1.00431107e+00
   5.65767394e-01 -4.62746495e-01  1.02835776e+00 -8.22319358e-01
  -1.74427201e-01 -7.10122686e-01  4.22352040e-01  1.19922572e+00
  -1.96434851e-01  1.49853557e+00  1.39314880e+00 -8.91255093e-01
  -3.01684210e-01 -1.48602593e-01 -2.47066583e-01 -1.19522493e-01
   1.41514901e+00  1.33169678e+00  3.41125749e-02 -4.78471139e-01
  -6.90400329e-04 -3.66202543e-01 -4.29749596e-01 -1.44341489e-01
  -1.94328272e-01 -4.72417464e-01 -4.74561378e-01 -4.42684243e-01
  -4.38222532e-01  4.58958215e-01 -4.12393442e-01 -6.49344383e-01
  -6.39306412e-01 -7.79318072e-01 -6.61422468e-01 -6.55720151e-01
  -6.39359641e-02 -4.58609061e-01  8.58983363e-02  7.26354913e-01
   8.85377799e-01 -4.92260531e-02 -1.54927811e-01 -7.09772525e-01
  -9.17416272e-01  6.70468142e-01 -5.12708251e-01 -8.87588695e-01
  -1.99177242e-01 -3.17475188e-01 -3.78164817e-01  6.21198252e-02
  -1.07335737e-01 -1.73085469e-01 -2.06712609e-01 -6.49870889e-01
   5.47236860e-02 -7.98858964e-01  1.02959504e-01 -2.26932480e-01
  -4.13507124e-01 -9.24682676e-01 -1.93319902e-01 -5.80753352e-01
  -5.81798607e-02  3.53728167e-02 -8.57125745e-01 -4.98253123e-02
  -7.23355255e-01  1.57515820e-01  4.08317478e-02  1.74244074e+00
  -8.57102799e-01  2.25073702e+00  1.75060600e+00 -1.04724393e+00
  -2.38022449e-02  8.15599972e-01 -1.50467968e+00 -5.16624713e-01
   2.74502833e-02  2.45690041e-01  5.00488487e-01  1.33699362e-01
   1.67316102e-01 -3.65794362e-01  4.31689364e-02  1.80845063e+00]
 [-2.56631489e-01 -1.27399288e+00 -1.25081974e-01 -5.32753267e-01
  -4.32260308e-01  1.03366411e-01  6.39913531e-02  2.26765473e-01
  -2.64095457e-01 -1.14439278e-01 -3.39022484e-01 -1.98207430e-01
  -3.55773281e-01 -2.75916919e-01 -8.24202677e-01 -7.07952109e-01
  -2.94897322e-01 -4.62746495e-01  1.31572608e+00  8.58060520e-01
  -1.65664385e+00  1.44697584e-01 -4.36731128e-01  2.95347135e+00
   1.11490360e+00  2.41216432e+00  2.75509710e+00 -1.27946552e+00
  -8.95962614e-01 -7.30274904e-01 -2.92792475e-01 -1.00744129e+00
  -2.45889096e-01  1.88925642e-02 -3.34063200e-01 -4.34775443e-02
  -2.30133443e-01  9.12140609e-01  1.23003039e+00  1.19595811e+00
   1.03694060e+00  2.91160291e-01  3.71914503e-01  2.75586410e-01
```

II. Visualize the data distribution in 2D using PCA

Principal Component Analysis is a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving variance. So, we reduced the data to 2 components, which is suitable for 2D visualization

```
pca_2d = PCA(n_components=2, random_state=42)
data_pca_2d = pca_2d.fit_transform(data_scaled)
print("\nDataset shape after 2D PCA:", data_pca_2d.shape)
print("Explained variance ratio:", pca_2d.explained_variance_ratio_)
```

Based on the 2 principal components, we plot the 2D PCA visualization for a dataset.



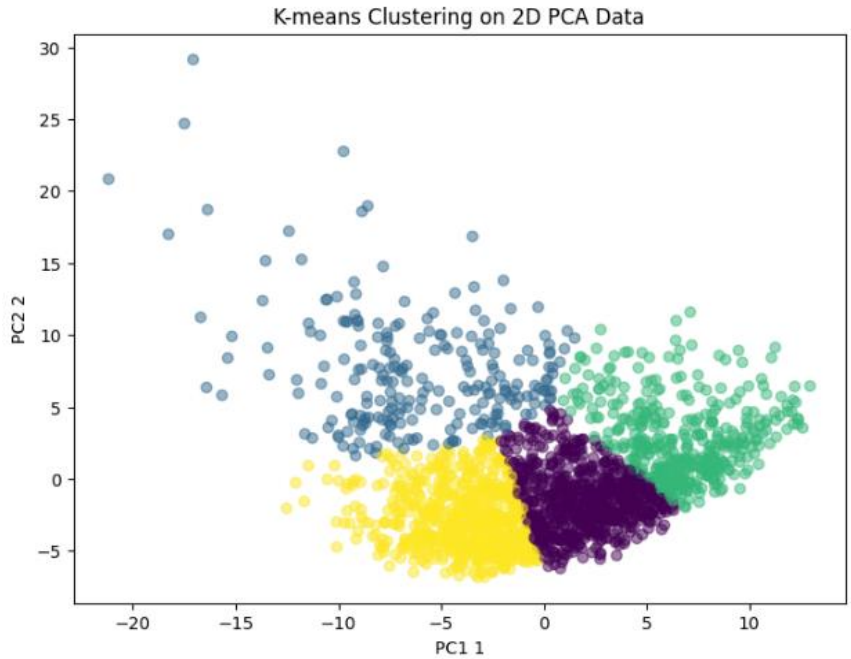
III. Apply k-means

We choose k = 4 to apply the k-means:

```
k = 4

# K-means on the original high-dimensional data
kmeans_orig = KMeans(n_clusters=k, random_state=42)
labels_orig = kmeans_orig.fit_predict(data_scaled)
sil_orig = silhouette_score(data_scaled, labels_orig)
print("\nSilhouette Score (Original high-dimensional data):", sil_orig)

# K-means on the 2D PCA-reduced data
kmeans_pca = KMeans(n_clusters=k, random_state=42)
labels_pca = kmeans_pca.fit_predict(data_pca_2d)
sil_pca = silhouette_score(data_pca_2d, labels_pca)
print("Silhouette Score (PCA 2D data):", sil_pca)
```



- Before dimensionality reduction, the clustering of the high-dimensional data is performed on all features. Although it may capture complex relationships, some redundant features might affect the quality of the cluster.
- After being dimensionality reduced to 2D through PCA, the majority of variance is preserved, so the clustering performance can remain in the original data. The 2D PCA graph also offers an intuitive visualization.

IV. Select a random subspace of the dataset

We choose randomly 10 features from the dataset

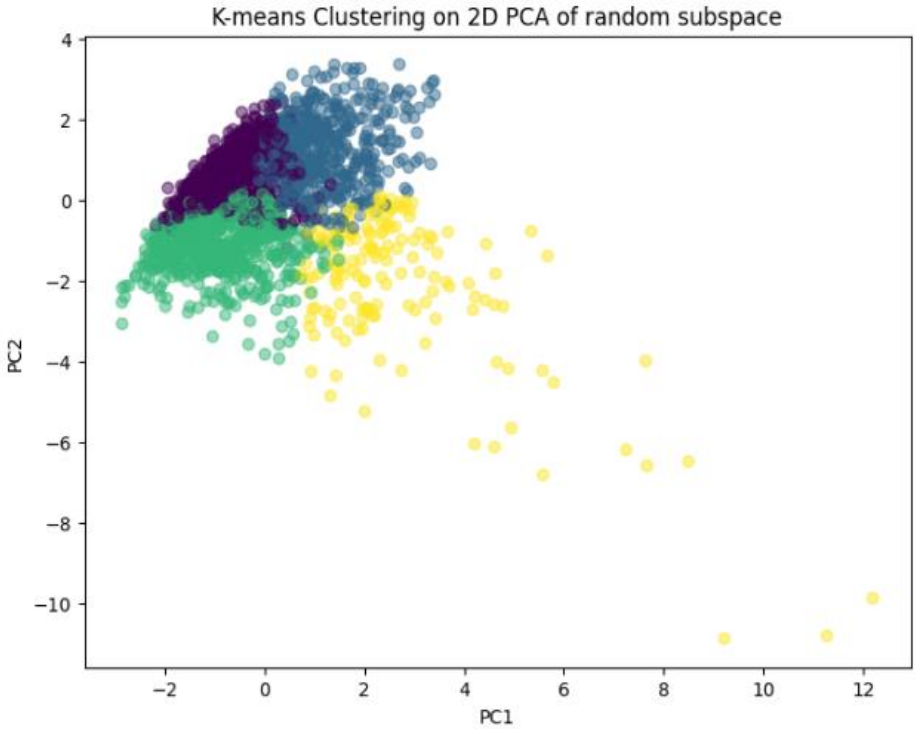
```
num_features = 10
random_features = np.random.choice(df.columns, size=num_features, replace=False)
print("\nRandom features:", random_features)
data_subspace = df[random_features]
```

We standardize the data and apply k-means to the subspace.

```
data_subspace_scaled = scaler.fit_transform(data_subspace)
print("Subspace dataset shape after scaling:", data_subspace_scaled.shape)
print(data_subspace_scaled[:5])
```

```
kmeans_sub = KMeans(n_clusters=k, random_state=42)
labels_sub = kmeans_sub.fit_predict(data_subspace_scaled)
sil_sub = silhouette_score(data_subspace_scaled, labels_sub)
```

We reduce the subspace data for 2D to plot visualization.



- Clustering on a randomly selected subspace helps gauge the importance of the full feature set. The lower the Silhouette score in the random subspace, the more features are needed for the missing clustering.